# Software Lab Report

## Assignment 1: Infrastructure as Code using Terraform

---

## 1. Problem Statement

Write a Terraform configuration using the local provider to create a file named `hello.txt` and store a message inside it using a variable. Display the file path as output.

---

## 2. Objective

To understand the basic Terraform workflow and learn how to create and manage a simple resource using Infrastructure as Code (IaC).

---

## 3. Explanation

Terraform is an Infrastructure as Code tool that allows users to define resources in configuration files instead of manually creating them.

In this assignment:

- The **local provider** is used to create a file on the system
- A **resource** (`local_file`) is defined to generate `hello.txt`
- A **variable** is used to make the message dynamic
- An **output** is used to display the file path

Terraform follows a standard workflow:

- **init** → Initializes the project
- **plan** → Shows changes before execution
- **apply** → Executes and creates the resource

---

# 4. Implementation

### main.tf

```
provider "local" {}

resource "local_file" "hello_file" {
  filename = "hello.txt"
  content  = var.message
}
```

### variables.tf

```
variable "message" {
  description = "Message to write into the file"
  type        = string
  default     = "Hello from Terraform"
}
```

### outputs.tf

```
output "file_path" {
  description = "Path of the created file"
  value       = local_file.hello_file.filename
}
```

---

# 5. Commands Executed

```
terraform init
terraform plan
terraform apply
```

---

# 6. Output

### Terraform Execution:

```
mukesh@Mukeshsai:~$ terraform -v;
Terraform v1.14.7
on linux_amd64
mukesh@Mukeshsai:~$ sudo apt install git -y;
git --version;
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
git version 2.43.0
mukesh@Mukeshsai:~$ mkdir -p terraform-lab/day1;
cd terraform-lab/day1;
```

```
mukesh@Mukeshsai:~/terraform-lab/day1$ terraform init;
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
mukesh@Mukeshsai:~/terraform-lab/day1$ terraform plan;

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.hello_file will be created
  + resource "local_file" "hello_file" {
      + content              = "Hello from Terraform"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "hello.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + file_path = "hello.txt"

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply"
now.
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + file_path = "hello.txt"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.hello_file: Creating...
local_file.hello_file: Creation complete after 0s [id=e4d8040f47c75fe38d88315d46db094b4736facd]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

file_path = "hello.txt"
```

```
mukesh@Mukeshsai:~/terraform-lab/day1$ ls;
hello.txt  main.tf  outputs.tf  terraform.tfstate  variables.tf
mukesh@Mukeshsai:~/terraform-lab/day1$ cat hello.txt;
Hello from Terraformmukesh@Mukeshsai:~/terraform apply -var="message=Hello Mukesh from WSL";lo Mukesh from WSL";
local_file.hello_file: Refreshing state... [id=e4d8040f47c75fe38d88315d46db094b4736facd]
```

```
Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.hello_file: Destroying... [id=e4d8040f47c75fe38d88315d46db094b4736facd]
local_file.hello_file: Destruction complete after 0s
local_file.hello_file: Creating...
local_file.hello_file: Creation complete after 0s [id=72e99dbd249a407482058ece1d96c116af2da9d9]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

file_path = "hello.txt"
```

```
mukesh@Mukeshsai:~/terraform-lab/day1$ cat hello.txt;
Hello Mukesh from WSLmukesh@Mukeshsai:~/terraform-lab/day1$
```

- File created: `hello.txt`
- File content: **Hello from Terraform**
- Output displayed: `file_path = hello.txt`

---

# 7. Result

The Terraform configuration was successfully executed, and the file was created using IaC principles.

---

## 8. Conclusion

This experiment demonstrates how Terraform automates infrastructure creation. It highlights the importance of variables, outputs, and the Terraform workflow in managing resources efficiently.

---

# Assignment 2: Variables and Multiple Configurations using Terraform

---

## 1. Problem Statement

Modify a Terraform configuration to use variables for filename and content, and execute it using different variable files (`dev.tfvars` and `prod.tfvars`) to simulate multiple environments.

---

## 2. Objective

To understand how Terraform variables and variable files can be used to create flexible and environment-specific configurations.

---

## 3. Explanation

Terraform allows parameterization of configurations using **input variables**, which makes the code reusable and dynamic.

In this assignment:

- The **filename** and **message** are defined as variables
- Separate variable files (`.tfvars`) are created for different environments
- The same Terraform configuration is reused for both **development** and **production**

This approach avoids duplication and enables environment-based infrastructure management.

---

# 4. Implementation

### main.tf

```
provider "local" {}

resource "local_file" "env_file" {
  filename = var.filename
  content  = var.message
}
```

---

### variables.tf

```
variable "filename" {
  description = "Name of the file"
  type        = string
}

variable "message" {
  description = "Content of the file"
  type        = string
}
```

---

### dev.tfvars

```
filename = "dev.txt"
message  = "Development environment"
```

---

### prod.tfvars

```
filename = "prod.txt"
message  = "Production environment"
```

---

# 5. Commands Executed

```
terraform init
terraform apply -var-file="dev.tfvars"
```

terraform apply -var-file="prod.tfvars"

---

# 6. Output

## Terraform Execution:

```
mukesh@Mukeshsai:~/terraform-lab/day1$ cd ~/terraform-lab;$ cd ~/terraform-lab;
mkdir -p day2;
cd day2;
mukesh@Mukeshsai:~/terraform-lab/day2$ nano main.tf;
mukesh@Mukeshsai:~/terraform-lab/day2$ nano variables.tf;
mukesh@Mukeshsai:~/terraform-lab/day2$ nano dev.tfvars;
mukesh@Mukeshsai:~/terraform-lab/day2$ nano prod.tfvars;
mukesh@Mukeshsai:~/terraform-lab/day2$ terraform init;
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
mukesh@Mukeshsai:~/terraform-lab/day2$ terraform apply -var-file="dev.tfvars";
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.env_file: Creating...
local_file.env_file: Creation complete after 0s [id=e99a087024263f55df4075593a3e45d2efec12e1]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
mukesh@Mukeshsai:~/terraform-lab/day2$ ls;
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
mukesh@Mukeshsai:~/terraform-lab/day2$ ls;
cat dev.txt;
dev.tfvars  dev.txt  main.tf  prod.tfvars  terraform.tfstate  variables.tf
Development environmentmukesh@Mukeshsai:~/terraform-lab/day2$ terraform apply -var-file="prod.tfvars";
local_file.env_file: Refreshing state... [id=e99a087024263f55df4075593a3e45d2efec12e1]
```

```
  # local_file.env_file must be replaced
-/+ resource "local_file" "env_file" {
      ~ content              = "Development environment" -> "Production environment" # forces replacement
      ~ content_base64sha256 = "it3WMaq7UEHIr0dilz8MDZk7t5WxfulCzDSkG1uX4Gc=" -> (known after apply)
      ~ content_base64sha512 = "jxCXEZykFErEhs7P3g3Arnl/8JqQ9dtQ3uXxGlImd5NrMSYlm3c2cNGqgri95MA2XXEeNOqCk7mlY5PqtxENdg==" -> (known after apply)
      ~ content_md5          = "6b3b0440b835d901050393db393ca743" -> (known after apply)
      ~ content_sha1         = "e99a087024263f55df4075593a3e45d2efec12e1" -> (known after apply)
      ~ content_sha256       = "8addd631aabb5041c8af4762973f0c0d993bb795b17ee942cc34a41b5b97e067" -> (known after apply)
      ~ content_sha512       = "8f1097119ca4144ac486cecfde0dc0ae797ff09a90f5db50dee5f11a522677936b3126259b773670d1aa82b8bde4c0365d711e34ea8293b9
a56393eab7110d76" -> (known after apply)
      ~ filename             = "dev.txt" -> "prod.txt" # forces replacement
      ~ id                   = "e99a087024263f55df4075593a3e45d2efec12e1" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.env_file: Destroying... [id=e99a087024263f55df4075593a3e45d2efec12e1]
local_file.env_file: Destruction complete after 0s
local_file.env_file: Creating...
local_file.env_file: Creation complete after 0s [id=56223686673ea90d19b3ea6af806ce8932aa208a]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

```
mukesh@Mukeshsai:~/terraform-lab/day2$ ls;
cat prod.txt;
dev.tfvars  main.tf  prod.tfvars  prod.txt  terraform.tfstate  terraform.tfstate.backup  variables.tf
Production environmentmukesh@Mukeshsai:~/terraform-lab/day2$ =
```

- File created (Dev): `dev.txt` → **Development environment**
- File created (Prod): `prod.txt` → **Production environment**

---

# 7. Result

The Terraform configuration was successfully executed using different variable files, and environment-specific files were created.

---

# 8. Conclusion

This experiment demonstrates how Terraform variables and `.tfvars` files enable reusable and scalable configurations. It highlights how the same code can be used across multiple environments efficiently.

---

# Assignment 3: Managing Containers with Terraform

---

## 1. Problem Statement

Configure Terraform to use the Docker provider to pull the nginx image, create a container, map container port 80 to host port 8080, and display the service URL.

---

## 2. Objective

To learn how Terraform can be used to provision and manage Docker containers and deploy services using Infrastructure as Code.

---

## 3. Explanation

Terraform supports multiple providers, including Docker, which enables users to manage containers programmatically.

In this assignment:

- The **Docker provider (kreuzwerker/docker)** is configured
- The **nginx image** is pulled from Docker Hub
- A **Docker container** is created using the image
- Port **80 (container)** is mapped to **8080 (host)**
- An **output variable** is used to display the service URL

This demonstrates how Terraform can automate container deployment and service exposure.

---

## 4. Implementation

### main.tf

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
```

```
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "nginx_container" {
  name  = "nginx_container"
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 8080
  }
}
```

---

### outputs.tf

```
output "service_url" {
  description = "URL to access nginx service"
  value       = "http://localhost:8080"
}
```

---

## 5. Commands Executed

terraform init
terraform apply

---

## 6. Output

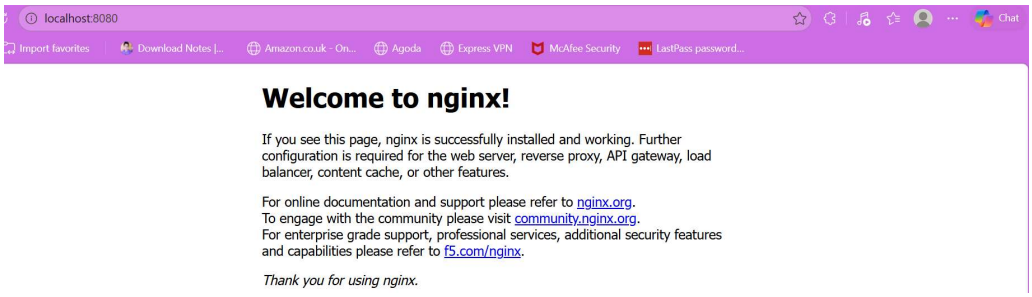
### Terraform Execution and Service Output:

```
mukesh@Mukeshsai:~/terraform-lab/day3$ terraform init -upgrade;
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Installing kreuzwerker/docker v3.6.2...
- Installed kreuzwerker/docker v3.6.2 (self-signed, key ID BD080C4571C6104C)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

```
mukesh@Mukeshsai:~/terraform-lab/day3$ terraform apply;

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
```

```
      + ports {
          + external = 8080
          + internal = 80
          + ip       = "0.0.0.0"
          + protocol = "tcp"
        }
    }

  # docker_image.nginx will be created
  + resource "docker_image" "nginx" {
      + id          = (known after apply)
      + image_id    = (known after apply)
      + name        = "nginx:latest"
      + repo_digest = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + service_url = "http://localhost:8080"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_image.nginx: Creating...
docker_image.nginx: Still creating... [00m09s elapsed]
docker_image.nginx: Still creating... [00m19s elapsed]
docker_image.nginx: Still creating... [00m29s elapsed]
docker_image.nginx: Still creating... [00m38s elapsed]
docker_image.nginx: Creation complete after 45s [id=sha256:1a1e6313642049664ed59099f3ca37a672486049af68395c62dd885a9bf4de35nginx:latest]
docker_container.nginx_container: Creating...
docker_container.nginx_container: Creation complete after 1s [id=d6abd85fb1860984bc5699a5111053cd0a2985998022be5d261541a760a92aec]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

service_url = "http://localhost:8080"
```

```
mukesh@Mukeshsai:~/terraform-lab/day3$ docker ps;
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                    NAMES
d6abd85fb186   1a1e63136420   "/docker-entrypoint.…"   6 seconds ago   Up 5 seconds   0.0.0.0:8080->80/tcp     nginx_container
mukesh@Mukeshsai:~/terraform-lab/day3$
```



- Docker container successfully created
- nginx service running at: **http://localhost:8080**
- Default nginx welcome page displayed

---

## 7. Result

The nginx container was successfully created and managed using Terraform, and the service was accessible through the mapped port.

---

## 8. Conclusion

This experiment demonstrates how Terraform can be used to automate container provisioning and manage services efficiently. It highlights the power of Infrastructure as Code in simplifying deployment and ensuring consistency.

---

# Assignment 4: Terraform Modules

---

## 1. Problem Statement

Create a reusable Terraform module by moving the container creation logic from Assignment 3 into a module. The module should accept variables and be used to create two nginx containers running on different ports (8081 and 8082).

---

## 2. Objective

To understand how Terraform modules help in structuring configurations and creating reusable infrastructure components.

---

## 3. Explanation

Terraform modules allow grouping of resources into reusable components. Instead of writing the same configuration multiple times, modules help in improving code reusability and maintainability.

In this assignment:

- The Docker container logic from Assignment 3 is moved into a module
- The module accepts variables such as **container name, port, and image name**
- The root configuration calls the module twice to create two containers
- Each module instance runs on a different port (8081 and 8082)

Additionally, the Docker provider (`kreuzwerker/docker`) is explicitly defined inside the module to avoid provider resolution errors.

---

# 4. Implementation

## Directory Structure

```
terraform-lab/day4
│
├── main.tf
└── modules/
    └── nginx_container/
        ├── main.tf
        └── variables.tf
```

---

## Module: main.tf

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

resource "docker_image" "nginx" {
  name = var.image_name
}
```

```
resource "docker_container" "nginx_container" {
  name  = var.container_name
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = var.container_port
  }
}
```

---

## Module: variables.tf

```
variable "container_name" {
  type = string
}

variable "container_port" {
  type = number
}

variable "image_name" {
  type = string
}
```

---

## Root: main.tf

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

module "nginx1" {
  source = "./modules/nginx_container"

  container_name = "nginx_container_1"
```

```
  container_port = 8081
  image_name     = "nginx:latest"
}

module "nginx2" {
  source = "./modules/nginx_container"

  container_name = "nginx_container_2"
  container_port = 8082
  image_name     = "nginx:latest"
}
```

---

## 5. Commands Executed

terraform init
terraform apply
Docker ps

---

## 6. Output

### Terraform Execution and Running Containers:

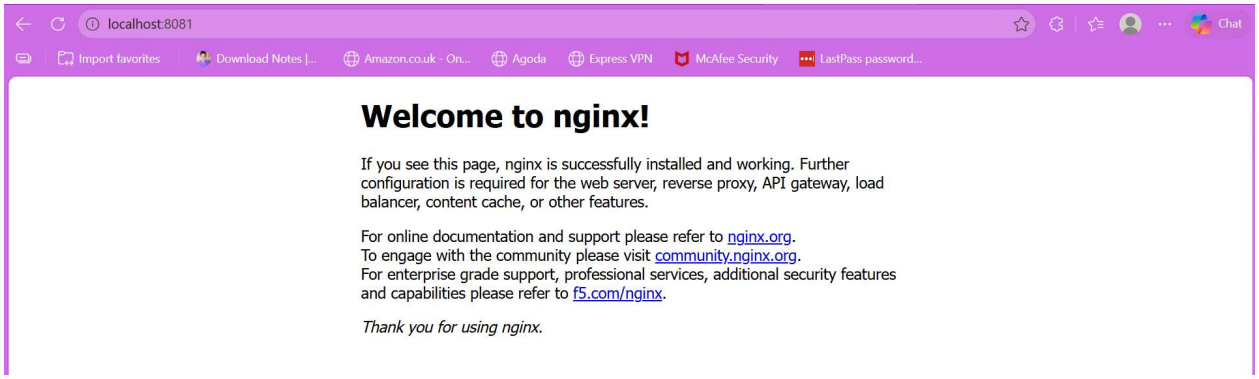
```
mukesh@Mukeshsai:~/terraform-lab/day4$ terraform init -upgrade;
Initializing the backend...
Upgrading modules...
- nginx1 in modules/nginx_container
- nginx2 in modules/nginx_container
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Installing kreuzwerker/docker v3.6.2...
- Installed kreuzwerker/docker v3.6.2 (self-signed, key ID BD080C4571C6104C)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

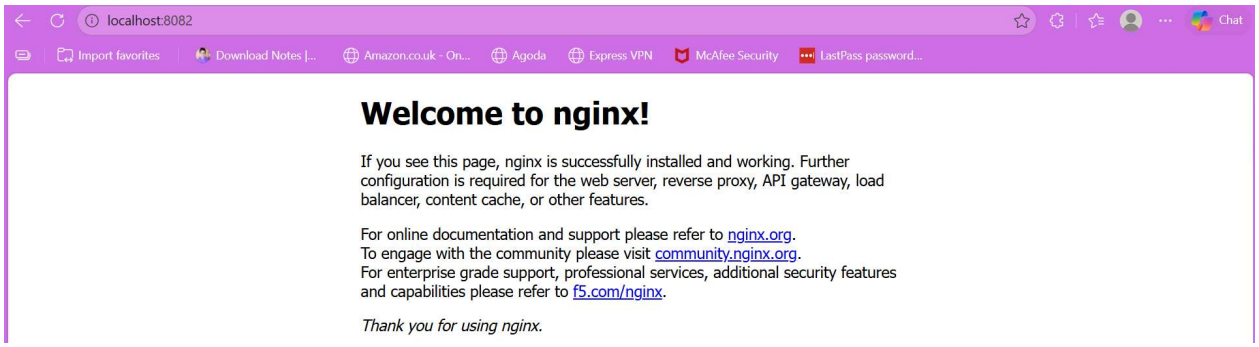
```
mukesh@Mukeshsai:~/terraform-lab/day4$ terraform apply;

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
```

```
      + ports {
          + external = 8081
          + internal = 80
          + ip       = "0.0.0.0"
          + protocol = "tcp"
        }
    }

  # module.nginx1.docker_image.nginx will be created
  + resource "docker_image" "nginx" {
      + id          = (known after apply)
      + image_id    = (known after apply)
      + name        = "nginx:latest"
      + repo_digest = (known after apply)
    }
```

```
      + ports {
          + external = 8082
          + internal = 80
          + ip       = "0.0.0.0"
          + protocol = "tcp"
        }
    }

  # module.nginx2.docker_image.nginx will be created
  + resource "docker_image" "nginx" {
      + id          = (known after apply)
      + image_id    = (known after apply)
      + name        = "nginx:latest"
      + repo_digest = (known after apply)
    }

Plan: 4 to add, 0 to change, 0 to destroy.
```

```
  Enter a value: yes

module.nginx2.docker_image.nginx: Creating...
module.nginx1.docker_image.nginx: Creating...
module.nginx2.docker_image.nginx: Creation complete after 0s [id=sha256:1a1e6313642049664ed59099f3ca37a672486049af68395c62dd885a9bf4de35nginx:la
test]
module.nginx1.docker_image.nginx: Creation complete after 0s [id=sha256:1a1e6313642049664ed59099f3ca37a672486049af68395c62dd885a9bf4de35nginx:la
test]
module.nginx2.docker_container.nginx_container: Creating...
module.nginx1.docker_container.nginx_container: Creating...
module.nginx1.docker_container.nginx_container: Creation complete after 1s [id=d115a5c104318ac8194bcc941ab4b435f45d7cb40825c8d0f22cfec29731fa2f]
module.nginx2.docker_container.nginx_container: Creation complete after 1s [id=29a66a018dd13cb33ee1cd6cda496905ea87c3f618af5015fd2c166676690a27]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
mukesh@Mukeshsai:~/terraform-lab/day4$ docker ps;
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                   NAMES
d115a5c10431   1a1e63136420   "/docker-entrypoint.…"   23 seconds ago   Up 23 seconds   0.0.0.0:8081->80/tcp    nginx_container_1
29a66a018dd1   1a1e63136420   "/docker-entrypoint.…"   23 seconds ago   Up 23 seconds   0.0.0.0:8082->80/tcp    nginx_container_2
d6abd85fb186   1a1e63136420   "/docker-entrypoint.…"   17 minutes ago   Up 17 minutes   0.0.0.0:8080->80/tcp    nginx_container
mukesh@Mukeshsai:~/terraform-lab/day4$
```




- Two Docker containers successfully created
- nginx services running at:
  - **http://localhost:8081**
  - **http://localhost:8082**
- Both ports display nginx welcome page

---

# 7. Result

The Terraform module was successfully created and reused to deploy two nginx containers on different ports.

---

# 8. Conclusion

This experiment demonstrates the use of Terraform modules to create reusable and structured configurations. It highlights how modular design improves scalability, maintainability, and reduces code duplication.

---

# Assignment 5: Automation and Validation using Terraform

---

## 1. Problem Statement

Create a CI workflow that automatically runs Terraform commands (`fmt`, `validate`, and `plan`) whenever code is pushed to the repository.

---

## 2. Objective

To understand how Infrastructure as Code can be validated automatically using Continuous Integration (CI) and ensure code quality using GitHub Actions.

---

## 3. Explanation

Continuous Integration (CI) is a development practice where code changes are automatically tested and validated when pushed to a repository.

In this assignment:

- A **GitHub Actions workflow** is created
- The workflow runs on every push to the `main` branch
- It executes Terraform commands:
  - **terraform fmt** → checks code formatting
  - **terraform validate** → verifies syntax correctness
  - **terraform plan** → previews infrastructure changes

Since the project contains multiple folders (day1–day4), a **matrix strategy** is used to run Terraform commands in each directory.

Additionally, for **day2**, a `.tfvars` file is passed to avoid interactive input during execution.

---

# 4. Implementation

## Workflow File Location

.github/workflows/terraform-check.yml

---

## terraform-check.yml

```yaml
name: Terraform Check

on:
  push:
    branches:
      - main

jobs:
  terraform:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        dir: [day1, day2, day3, day4]

    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init
        working-directory: ./${{ matrix.dir }}

      - name: Terraform Format Check
        run: terraform fmt -check
        working-directory: ./${{ matrix.dir }}

      - name: Terraform Validate
        run: terraform validate
        working-directory: ./${{ matrix.dir }}

      - name: Terraform Plan
```

```
    run: |
      if [ "${{ matrix.dir }}" = "day2" ]; then
        terraform plan -var-file="dev.tfvars"
      else
        terraform plan
      fi
    working-directory: ./${{ matrix.dir }}
```

---

## 5. Commands Executed

```
git add .
git commit -m "Added Terraform CI workflow"
git push
```

---

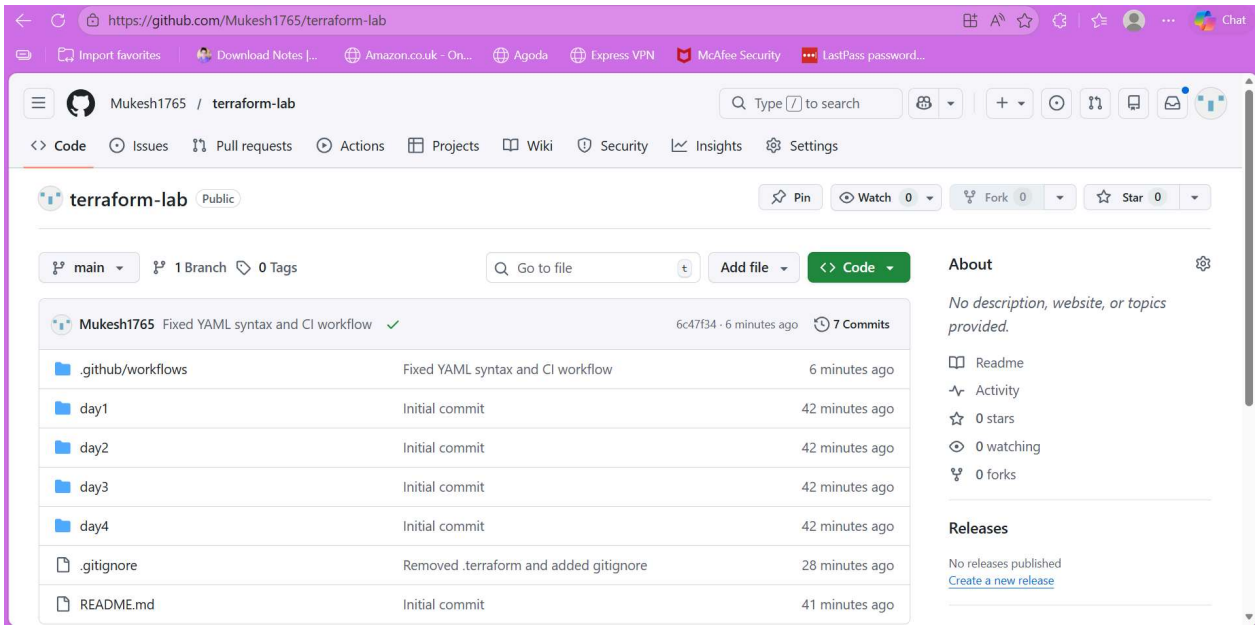
## 6. Output

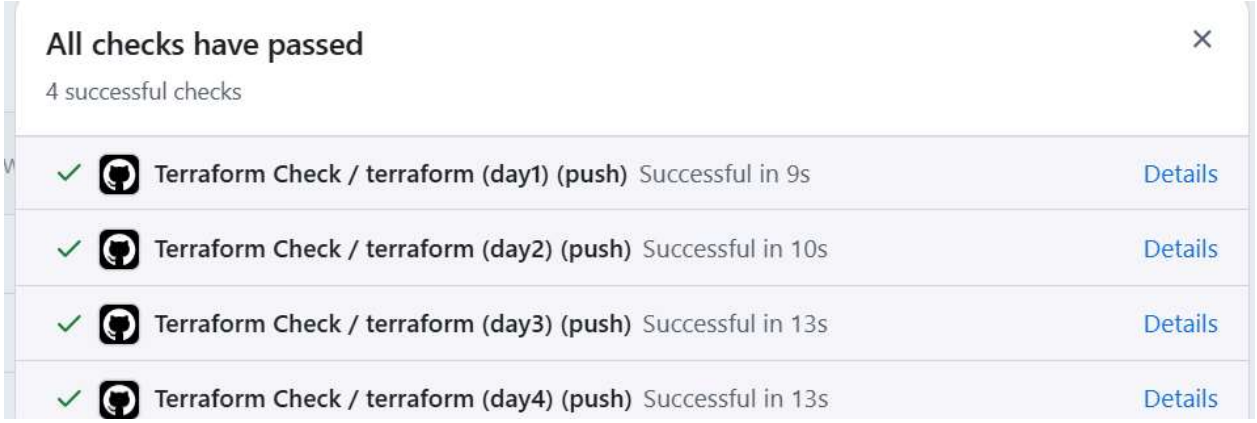
### GitHub Actions Workflow Execution:

```
mukesh@Mukeshsai:~/terraform-lab$ git push;
Username for 'https://github.com': Mukesh1765
Password for 'https://Mukesh1765@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 324 bytes | 324.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Mukesh1765/terraform-lab.git
   af05844..11a656c  main -> main
```

```
git add .;
git commit -m "Added Terraform CI workflow";
git push;
[main 7d97e6d] Added Terraform CI workflow
 1 file changed, 37 insertions(+)
 create mode 100644 .github/workflows/terraform-check.yml
```

```
mukesh@Mukeshsai:~/terraform-lab/.github/workflows$ gedit terraform-check.yml;
mukesh@Mukeshsai:~/terraform-lab/.github/workflows$ cd ~/terraform-lab;

git add .;
git commit -m "Fixed YAML syntax and CI workflow";
[main 6c47f34] Fixed YAML syntax and CI workflow
 1 file changed, 7 insertions(+), 8 deletions(-)
mukesh@Mukeshsai:~/terraform-lab$ git push;
Username for 'https://github.com': Mukesh1765
Password for 'https://Mukesh1765@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 474 bytes | 474.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Mukesh1765/terraform-lab.git
   41350dd..6c47f34  main -> main
mukesh@Mukeshsai:~/terraform-lab$
```



- Workflow triggered automatically on push
- Terraform checks executed for all directories
- All jobs completed successfully

---

## 7. Result

The CI workflow was successfully implemented. Terraform configurations were automatically validated on every push, ensuring correctness and consistency.

---

## 8. Conclusion

This experiment demonstrates how CI can be integrated with Terraform to automate validation of infrastructure code. It improves reliability, reduces errors, and ensures high-quality infrastructure deployment practices.

---

# Advanced Bonus: Dynamic Infrastructure Creation using Terraform

---

## 1. Problem Statement

Extend the module-based Terraform infrastructure to dynamically create multiple Docker containers using input configuration. Each container must run nginx, have a unique name, and use a specified port.

---

## 2. Objective

To learn how to dynamically generate infrastructure resources using Terraform expressions such as `for_each`, and avoid manual duplication of resource definitions.

---

## 3. Explanation

Terraform supports dynamic infrastructure creation using constructs like `for_each` and `count`. In this assignment, `for_each` is used to iterate over a list of container configurations.

- A variable `container_config` stores multiple container definitions
- Terraform automatically creates one container per configuration
- Each container is assigned a unique name and port
- When the configuration list changes, Terraform updates infrastructure accordingly

This approach ensures scalability and flexibility in infrastructure management.

---

## 4. Implementation

### main.tf

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "nginx_container" {
  for_each = {
    for container in var.container_config :
    container.name => container
  }

  name  = each.value.name
  image = docker_image.nginx.image_id

  ports {
    internal = 80
```

```
    external = each.value.port
  }
}
```

---

## variables.tf

```
variable "container_config" {
  description = "List of container configurations"
  type = list(object({
    name = string
    port = number
  }))
}
```

---

## terraform.tfvars

```
container_config = [
  {
    name = "web1"
    port = 8081
  },
  {
    name = "web2"
    port = 8082
  },
  {
    name = "web3"
    port = 8083
  }
]
```

---

## outputs.tf

```
output "container_urls" {
  value = [
    for c in var.container_config :
    "http://localhost:${c.port}"
  ]
}
```

---

# 5. Commands Executed

terraform init
terraform plan
terraform apply

---

# 6. Output

## Terraform Plan (Dynamic Resource Creation):

```
mukesh@Mukeshsai:~/terraform-lab/day5$ terraform init;
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Installing kreuzwerker/docker v3.6.2...
- Installed kreuzwerker/docker v3.6.2 (self-signed, key ID BD080C4571C6104C)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

```
mukesh@Mukeshsai:~/terraform-lab/day5$ terraform plan;

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
```

```
  + ports {
      + external = 8081
      + internal = 80
      + ip       = "0.0.0.0"
      + protocol = "tcp"
    }
```

```
  + ports {
      + external = 8082
      + internal = 80
      + ip       = "0.0.0.0"
      + protocol = "tcp"
    }
```

```
  + ports {
      + external = 8083
      + internal = 80
      + ip       = "0.0.0.0"
      + protocol = "tcp"
    }
```

```
mukesh@Mukeshsai:~/terraform-lab/day5$ terraform apply;

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
```

```
Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + container_urls = [
      + "http://localhost:8081",
      + "http://localhost:8082",
      + "http://localhost:8083",
    ]

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_image.nginx: Creating...
docker_image.nginx: Creation complete after 0s [id=sha256:1a1e6313642049664ed59099f3ca37a672486049af68395c62dd885a9bf4de35nginx:latest]
docker_container.nginx_container["web3"]: Creating...
docker_container.nginx_container["web1"]: Creating...
docker_container.nginx_container["web2"]: Creating...
docker_container.nginx_container["web1"]: Creation complete after 1s [id=9b096f558be64fd88e2046776e3e2b395b10ad166ffd86fb3d1447057a145dda]
docker_container.nginx_container["web3"]: Creation complete after 1s [id=cc4cf5c507104eb505053a673e8ec69683b4c1f11ed1240acb79918ffe139228]
docker_container.nginx_container["web2"]: Creation complete after 1s [id=49c82d9a515e27da849a26ef784aef185fa4ddd46e5c58bec764efd346b8e1fb]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

container_urls = [
  "http://localhost:8081",
  "http://localhost:8082",
  "http://localhost:8083",
]
```

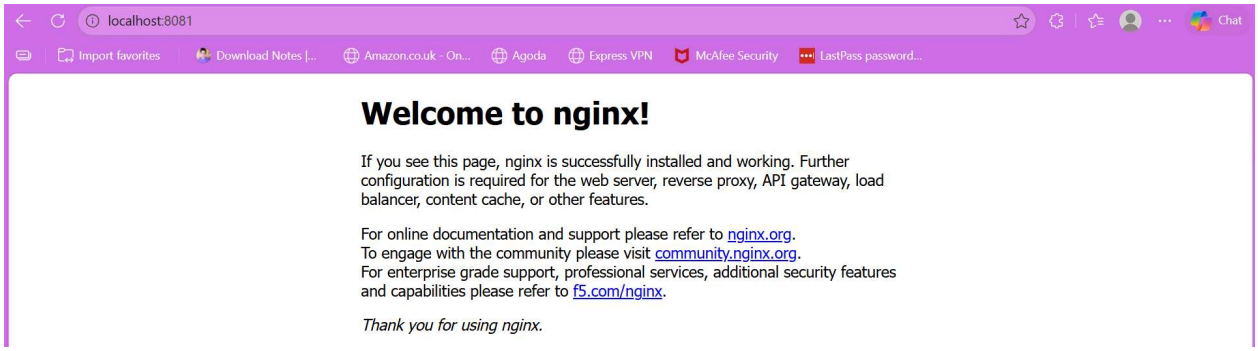
- Terraform plan shows creation of multiple containers:
  - docker_container.nginx_container["web1"]
  - docker_container.nginx_container["web2"]
  - docker_container.nginx_container["web3"]

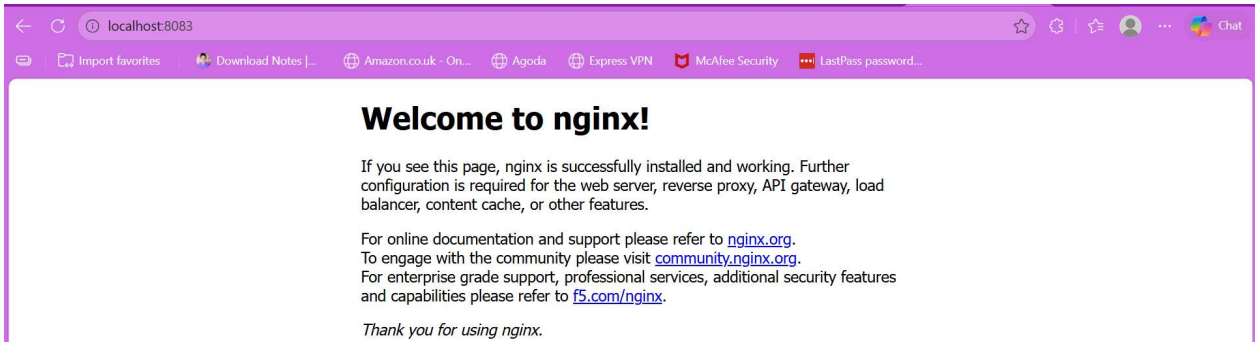
---

## Running Containers:

```
mukesh@Mukeshsai:~/terraform-lab/day5$ docker ps;
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                    NAMES
cc4cf5c50710   1a1e63136420   "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8083->80/tcp     web3
49c82d9a515e   1a1e63136420   "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8082->80/tcp     web2
9b096f558be6   1a1e63136420   "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8081->80/tcp     web1
d6abd85fb186   1a1e63136420   "/docker-entrypoint.…"   2 hours ago      Up 2 hours      0.0.0.0:8080->80/tcp     nginx_container
```

```
mukesh@Mukeshsai:~/terraform-lab/day5$ terraform output;
container_urls = [
  "http://localhost:8081",
  "http://localhost:8082",
  "http://localhost:8083",
]
mukesh@Mukeshsai:~/terraform-lab/day5$
```






- Multiple containers successfully created
- Services accessible at:
  - http://localhost:8081
  - http://localhost:8082
  - http://localhost:8083

---

### Terraform Output:

```
container_urls = [
  "http://localhost:8081",
  "http://localhost:8082",
  "http://localhost:8083"
]
```

---

# 7. Result

Terraform successfully created multiple Docker containers dynamically using input configuration. The infrastructure scaled automatically without modifying resource definitions.

---

## 8. Conclusion

This experiment demonstrates how Terraform can dynamically manage infrastructure using `for_each`. It eliminates code duplication and enables scalable, maintainable infrastructure design, which is widely used in real-world DevOps practices.

---